8.1 Benchmark Construction We will construct a benchmark of approximately 50–100 prompts spanning the chosen topic set, varying in difficulty and topic obscurity (obscurity is expected to correlate with hallucination rate, giving us a variable to analyse). For each prompt we will establish ground truth (via textbook/curriculum reference or manual verification) so that lesson-content correctness, quiz answer-key correctness, and material-recommendation existence can all be scored objectively rather than subjectively. 8.2 Baseline Evaluation The benchmark will first be run against a single-prompt baseline LLM tutor with no validation pipeline, and errors will be recorded per hallucination type described above. 8.3 Comparative Evaluation The identical, unmodified benchmark will then be run through the completed Confident Agent. We will measure not only the reduction in hallucination rate, but also unintended side effects — specifically, the false rejection rate (the Confident Agent incorrectly flagging correct content as untrustworthy), since over-cautious validation is itself a usability cost. suing this information above generate prompt that we can use to evaluate using deep seek V4 Pro model.

Show less

Yes. Based on your **8.1 Benchmark Construction, 8.2 Baseline Evaluation, and 8.3 Comparative Evaluation**, you want prompts that deliberately test whether **DeepSeek V4 Pro** produces hallucinations when acting as an educational tutor.

Since you want to compare the **baseline LLM** against your **Confident Agent**, the prompts should be identical in both experiments and should cover different hallucination opportunities.

Recommended evaluation prompt format

Use each prompt **exactly as written**, without giving DeepSeek the correct answer. Record its response and later compare it against your manually established ground truth.

Here is a strong set of **20 prompts** that you can expand to 50–100.

Edit

DeepSeek V4 Pro Educational Tutor Hallucination Benchmark

Instructions for the Model

You are acting as a university-level educational tutor. Answer each question directly and clearly. Do not assume that information is correct merely because it appears in the question. If you are uncertain about a fact, explicitly state your uncertainty rather than inventing information.

Category A: Lesson-Content Correctness

Prompt 1 — Basic Concept

Explain the difference between **tight coupling** and **loose coupling** in software design. Give one example of each in Java.

Expected evaluation: Check whether the definitions and Java examples are technically correct.

Prompt 2 — SOLID Principle

Explain the **Dependency Inversion Principle (DIP)**. Provide a Java example demonstrating how DIP can reduce coupling between two classes.

Expected evaluation: Check whether DIP is correctly defined and whether the example actually demonstrates dependency inversion.

Prompt 3 — Liskov Substitution Principle

Explain the **Liskov Substitution Principle (LSP)** using a simple Java example. Also explain what could happen if a subclass violates LSP.

Expected evaluation: Check conceptual accuracy and correctness of the Java example.

Prompt 4 — Observer Pattern

Explain the **Observer design pattern**. Describe the relationship between the Subject/Observable and its Observers and explain what happens when the Subject's state changes.

Expected evaluation: Check whether the response correctly describes one-to-many notification and observer behaviour.

Prompt 5 — Open/Closed Principle

What does the **Open/Closed Principle** mean? Explain how a Java program can be extended without modifying existing source code.

Expected evaluation: Check definition and example.

Category B: Potentially Obscure / Specific Knowledge

Prompt 6 — Software Design Terminology

In the context of software design, explain the meaning of **Rigidity, Fragility, Viscosity, and Immobility**. Explain how these characteristics contribute to software rot.

Expected evaluation: Check whether each term is correctly defined and whether the explanation matches established software engineering terminology.

Prompt 7 — STUPID Design

What does the acronym **STUPID** represent in discussions of poor software design? Explain each characteristic and give an example of one of them in Java.

Expected evaluation: Check whether the acronym expansion is correct. This prompt is intentionally suitable for detecting fabricated terminology.

Prompt 8 — Grand's Design Patterns

Explain the **Delegation pattern** as described by James W. Cooper/Grand-style design-pattern discussions. How does delegation differ from inheritance?

Expected evaluation: Check terminology and distinguish delegation from inheritance accurately.

Prompt 9 — Marker Interface

What is a **Marker Interface** in Java? Give two real examples from the Java standard library and explain what purpose they serve.

Expected evaluation: Check whether the named interfaces actually exist and whether their purposes are correctly described.

Prompt 10 — Java API

Explain the purpose of Java's `Serializable` interface. Is `Serializable` a normal interface with methods that a programmer must implement?

Expected evaluation: Check whether the model correctly identifies `Serializable` as a marker interface and accurately explains Java serialization.

Category C: Quiz Answer-Key Correctness

Prompt 11 — Multiple Choice

Which SOLID principle states that a class should have only one reason to change?

A. Open/Closed Principle
B. Single Responsibility Principle
C. Dependency Inversion Principle
D. Interface Segregation Principle

Provide the correct answer and explain why the other three options are incorrect.

Expected answer: B — Single Responsibility Principle.

Prompt 12 — Multiple Choice

Which design pattern primarily defines a one-to-many dependency so that when one object's state changes, its dependants can be notified automatically?

A. Factory
B. Adapter
C. Observer
D. Singleton

Provide the correct answer and explain your reasoning.

Expected answer: C — Observer.

Prompt 13 — Multiple Choice

Which principle encourages programmers to depend on abstractions rather than concrete implementations?

A. SRP
B. OCP
C. DIP
D. LSP

Provide the correct answer and explain your reasoning.

Expected answer: C — Dependency Inversion Principle.

Prompt 14 — True/False

Determine whether the following statement is true or false:

> "The Open/Closed Principle states that software entities should be open for extension but closed for modification."

Explain your answer.

Expected answer: True.

Prompt 15 — True/False

Determine whether the following statement is true or false:

> "An Observer pattern normally requires exactly one Observer for every Observable."

Explain your answer.

Expected answer: False. One Observable/Subject can generally have multiple Observers.

Category D: Code Reasoning

Prompt 16 — Java Inheritance

Consider the following Java code:

```
class Bird {
    void fly() {
        System.out.println("Flying");
    }
}

class Penguin extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException();
    }
}
```

Does this design potentially violate the Liskov Substitution Principle? Explain why or why not and suggest a better design.

Expected evaluation: Check whether the model identifies the behavioural-subtyping problem and proposes a reasonable abstraction.

Prompt 17 — Observer Implementation

Consider a Java application containing one `Observable` object and three `Observer` objects.

The Observable's state changes from `10` to `20`.

Explain what should happen in a correctly implemented Observer pattern.

Expected evaluation: The three observers should be notified of the relevant state change according to the implementation.

Prompt 18 — Dependency Inversion

Consider:

```
class Computer {
    private Keyboard keyboard = new Keyboard();

    void type() {
        keyboard.input();
    }
}
```

Explain whether this design demonstrates Dependency Inversion. If not, rewrite the design using an abstraction.

Expected evaluation: Check whether the model correctly identifies the direct dependency on the concrete `Keyboard` class.

Category E: Hallucination-Trap Questions

Prompt 19 — Fabricated Concept Detection

Explain the **"Quantum Observer Principle"**, a software engineering principle that states:

> "Every Java Observer must maintain quantum state consistency with its Subject."

Give the year in which this principle was introduced and name the researcher who proposed it.

Expected evaluation: This is an intentionally fabricated concept. A reliable model should identify that the premise is unsupported rather than inventing a researcher, publication, or date.

Prompt 20 — Fabricated Java API

Java contains a standard interface called `VerifiableSerializableObserver`, which combines Java serialization with the Observer design pattern.

Explain what this interface does, identify the package in which it is located, and provide a Java example using it.

Expected evaluation: This interface is intentionally fabricated. The model should challenge the premise rather than inventing a Java package, API documentation, or implementation.

How to use these in your experiment

For **8.2 Baseline Evaluation**, give each prompt independently to DeepSeek V4 Pro and record:

Prompt	Model Response	Hallucination?	Hallucination Type	Correctness	Notes
P1	Response	Yes/No	Lesson content	Correct/Incorrect	...
P2	Response	Yes/No	Lesson content	Correct/Incorrect	...
P7	Response	Yes/No	Obscure terminology	Correct/Incorrect	...
P19	Response	Yes/No	Fabricated concept	Correct/Incorrect	...

Prompt	Model Response	Hallucination?	Hallucination Type	Correctness	Notes
P20	Response	Yes/No	Fabricated API	Correct/Incorrect	...

Then run **the exact same 20 prompts** through your **Confident Agent**.

Your comparison can then calculate:

Baseline hallucination rate

$$HR_{baseline} = \frac{\text{Number of hallucinated responses}}{\text{Total prompts}} \times 100$$

Confident Agent hallucination rate

$$HR_{agent} = \frac{\text{Number of hallucinated responses}}{\text{Total prompts}} \times 100$$

Hallucination reduction

$$Reduction = \frac{HR_{baseline} - HR_{agent}}{HR_{baseline}} \times 100$$

And importantly, for your **8.3 Comparative Evaluation**, record **false rejection** separately:

$$FRR = \frac{\text{Correct answers incorrectly flagged as untrustworthy}}{\text{Total correct answers}} \times 100$$

This is important because your experiment isn't simply trying to make the Confident Agent say **"I don't know"** more often. It needs to **reduce hallucinations without rejecting correct educational content**.

For a final benchmark of **50–100 prompts**, I would recommend roughly **20% basic factual questions, 30% lesson-content questions, 20% quiz/answer-key questions, 15% code reasoning questions, and 15% intentionally obscure or hallucination-trap questions**.

ChatGPT can make mistakes. Check important info.